

This notebook generates the Yang Zhang volatility estimator for the CTA strategy

Motivation

The standard volatility estimator used in finance is the rolling standard deviation of close-to-close returns:

$$\sigma_{CC} = \text{Std} \left(\ln \frac{C_t}{C_{t-1}} \right)$$

While simple, this estimator ignores a large amount of information available within daily price bars. Specifically, it does not account for:

- Intraday price ranges (High-Low)
- Opening gaps
- Differences between overnight and intraday volatility

For futures markets, where large moves can occur outside regular trading hours due to macroeconomic announcements, geopolitical events, weather shocks, or inventory releases, relying solely on close-to-close returns may underestimate true market risk.

Yang-Zhang Volatility

The Yang-Zhang estimator (Yang & Zhang, 2000) is an OHLC-based volatility estimator designed to incorporate:

1. Overnight volatility
2. Intraday volatility
3. Intraday price range information

The estimator combines:

Overnight Return

$$r_t^o = \ln \left(\frac{O_t}{C_{t-1}} \right)$$

Intraday Return

$$r_t^c = \ln \left(\frac{C_t}{O_t} \right)$$

Rogers-Satchell Range Estimator

$$RS_t = \ln\left(\frac{H_t}{C_t}\right) \ln\left(\frac{H_t}{O_t}\right) + \ln\left(\frac{L_t}{C_t}\right) \ln\left(\frac{L_t}{O_t}\right)$$

The final Yang-Zhang variance estimate combines these three components:

$$\sigma_{YZ}^2 = \sigma_o^2 + k\sigma_c^2 + (1 - k)\sigma_{RS}^2$$

where:

- σ_o^2 = overnight variance
 - σ_c^2 = intraday variance
 - σ_{RS}^2 = Rogers-Satchell variance
 - k = weighting parameter determined by sample size
-

Advantages

Uses More Information

Unlike close-to-close volatility, Yang-Zhang incorporates:

- Open
- High
- Low
- Close

This allows the estimator to extract substantially more information from each trading day.

Captures Overnight Risk

Many commodity markets experience significant overnight moves due to:

- OPEC announcements
- USDA reports
- Weather developments
- Geopolitical events
- Central bank decisions

Yang-Zhang explicitly models overnight returns, while Parkinson and Garman-Klass estimators do not.

Lower Estimation Error

Empirical studies show that Yang-Zhang is significantly more efficient than traditional close-to-close volatility estimators.

Widely Used

Yang-Zhang is one of the most widely cited OHLC volatility estimators in both academia and quantitative trading.

Weaknesses

Requires OHLC Data

The estimator cannot be calculated from closing prices alone.

More Complex

Implementation is substantially more involved than rolling standard deviation or Parkinson volatility.

Still Backward Looking

Like all realized volatility estimators, Yang-Zhang describes past volatility rather than forecasting future volatility.

Sensitive to Data Quality

Incorrect or missing Open, High, or Low observations can materially distort estimates.

Relevance for This Project

This project studies systematic futures strategies across commodities, rates, equities, and FX.

Because the dataset contains complete OHLC data, Yang-Zhang volatility can be computed for every market in the universe.

Potential applications include:

- Volatility-scaled momentum signals
- Risk targeting
- Position sizing
- Portfolio optimization
- Regime detection

The estimator is particularly relevant for commodity futures, where overnight information often contains a substantial fraction of total volatility.

In [1]: *# Import packages*

```
import pandas as pd
import numpy as np
```

In [3]: *# Read*

```
folder_path = r"C:\Users\pcarg\OneDrive\Υπολογιστής\MFE UCLA\Projects\Commodities\D

open_prices = pd.read_parquet(f"{folder_path}\\open.parquet")
high_prices = pd.read_parquet(f"{folder_path}\\high.parquet")
low_prices = pd.read_parquet(f"{folder_path}\\low.parquet")
close_prices = pd.read_parquet(f"{folder_path}\\close.parquet")
volume = pd.read_parquet(f"{folder_path}\\volume.parquet")
```

In [5]: *# Check shape*

```
print(open_prices.shape)
print(close_prices.shape)

print("\nAssets:")
print(close_prices.columns.tolist())
```

(6475, 38)

(6475, 38)

Assets:

['WTI Crude', 'Brent', 'Natural Gas', 'RBOB', 'HeatingOil', 'Gold', 'Silver', 'Platinum', 'Palladium', 'Copper', 'Corn', 'Soybeans', 'Wheat', 'KC_Wheat', 'Rice', 'Oats', 'Coffee', 'Sugar', 'Cocoa', 'Cotton', 'OrangeJuice', 'LiveCattle', 'LeanHogs', 'FeederCattle', 'SP500', 'Nasdaq100', 'Dow', 'Russell2000', 'US_2Y', 'US_5Y', 'US_10Y', 'US_30Y', 'UltraBond', 'EURUSD', 'JPYUSD', 'GBPUSD', 'AUDUSD', 'CADUSD']

The estimator uses log prices so let's start from that.

In [7]: *# =====*
Log Prices
=====

```
log_o = np.log(open_prices)
log_h = np.log(high_prices)
log_l = np.log(low_prices)
log_c = np.log(close_prices)
```

=====
Overnight Returns
$r_o = \ln(O_t / C_{t-1})$
=====

```
r_o = log_o - log_c.shift(1)
```

=====
Intraday Returns

```

# r_c = ln(C_t / O_t)
# =====

r_c = log_c - log_o

# =====
# Rogers-Satchell Component
# =====

rs = (
    (log_h - log_c) * (log_h - log_o)
    +
    (log_l - log_c) * (log_l - log_o)
)

```

C:\Users\pcarg\anaconda3\Lib\site-packages\pandas\core\internals\blocks.py:393: RuntimeWarning: invalid value encountered in log
 result = func(self.values, **kwargs)

In [9]: *# Make a Yang-Zhang vol estimator*

```

def yang_zhang_volatility(
    open_prices,
    high_prices,
    low_prices,
    close_prices,
    window=63
):
    """
    Yang-Zhang volatility estimator.

    Parameters
    -----
    open_prices : pd.DataFrame
    high_prices : pd.DataFrame
    low_prices : pd.DataFrame
    close_prices : pd.DataFrame
    window : int
        Rolling window length.

    Returns
    -----
    pd.DataFrame
        Daily Yang-Zhang volatility estimate.
    """

    # Replace non-positive prices with NaN before taking logs
    def safe_log(df):
        return np.log(df.where(df > 0))

    log_o = safe_log(open_prices)
    log_h = safe_log(high_prices)
    log_l = safe_log(low_prices)
    log_c = safe_log(close_prices)

```

```

# Overnight returns
r_o = log_o - log_c.shift(1)

# Open-to-close returns
r_c = log_c - log_o

# Rogers-Satchell volatility component
rs = (
    (log_h - log_c) * (log_h - log_o)
    +
    (log_l - log_c) * (log_l - log_o)
)

# Rolling variances
sigma_o2 = r_o.rolling(window).var()

sigma_c2 = r_c.rolling(window).var()

sigma_rs = rs.rolling(window).mean()

# Yang-Zhang weighting parameter
k = 0.34 / (
    1.34 + (window + 1) / (window - 1)
)

# Yang-Zhang variance
yz_var = (
    sigma_o2
    + k * sigma_c2
    + (1 - k) * sigma_rs
)

# Daily volatility
yz_vol = np.sqrt(yz_var)

return yz_vol

```

Now use to compute the horizons we need.

```

In [12]: # Calculate

yz_vol_21 = yang_zhang_volatility(
    open_prices,
    high_prices,
    low_prices,
    close_prices,
    window=21
)

yz_vol_63 = yang_zhang_volatility(
    open_prices,
    high_prices,
    low_prices,
    close_prices,
    window=63
)

```

```
)

yz_vol_126 = yang_zhang_volatility(
    open_prices,
    high_prices,
    low_prices,
    close_prices,
    window=126
)

yz_vol_252 = yang_zhang_volatility(
    open_prices,
    high_prices,
    low_prices,
    close_prices,
    window=252
)
```

```
In [14]: # =====
# Annualized Versions
# =====

yz_vol_21_ann = yz_vol_21 * np.sqrt(252)
yz_vol_63_ann = yz_vol_63 * np.sqrt(252)
yz_vol_126_ann = yz_vol_126 * np.sqrt(252)
yz_vol_252_ann = yz_vol_252 * np.sqrt(252)
```

```
In [16]: # =====
# Sanity Checks
# =====

print(yz_vol_63["WTI Crude"].dropna().tail())

print("\nNumber of NaNs:")
print(yz_vol_63.isna().sum().sort_values(ascending=False).head())
```

```
Date
2026-06-02    0.064370
2026-06-03    0.064273
2026-06-04    0.064006
2026-06-05    0.063137
2026-06-07    0.053776
Name: WTI Crude, dtype: float64
```

```
Number of NaNs:
Russell2000    4295
Brent          2927
UltraBond      2539
Platinum       1544
Palladium      1340
dtype: int64
```

```
In [18]: # Compare with close-to-close volatility

log_returns = np.log(
    close_prices.where(close_prices > 0)
```

```

).diff()

cc_vol_63 = log_returns.rolling(63).std()

comparison = pd.DataFrame({
    "CC Vol": cc_vol_63["WTI Crude"],
    "YZ Vol": yz_vol_63["WTI Crude"]
})

comparison.tail()

```

Out[18]:

	CC Vol	YZ Vol
Date		
2026-06-02	0.054919	0.064370
2026-06-03	0.054976	0.064273
2026-06-04	0.054243	0.064006
2026-06-05	0.052394	0.063137
2026-06-07	0.052196	0.053776

In [20]: # Save

```

yz_features = pd.concat(
    {
        "yz_vol_21": yz_vol_21,
        "yz_vol_63": yz_vol_63,
        "yz_vol_126": yz_vol_126,
        "yz_vol_252": yz_vol_252,
    },
    axis=1
)

yz_features.to_parquet(
    r"C:\Users\pcarg\OneDrive\Υπολογιστής\MFE UCLA\Projects\Commodities\Data\yang_z
)

```